

SMART MCQ SOLVER CHALLENGE - TOP 3 ANSWER PREDICTION

Kaggle Competition Report | Latest Score: 0.75602 | Target Cutoff: 0.7300

PRINCE PATEL | 23F3002830 | IIT Madras

1. Abstract / Executive Summary

In this project, we built a complete machine learning pipeline for the **Smart MCQ Solver Challenge** on Kaggle. The goal is to predict the top 3 best option choices (A, B, C, D, E) for multiple-choice science questions evaluated using Mean Average Precision @ 3 (MAP@3).

We developed and evaluated three distinct models: (1) **Model 1: PyTorch OptionScorerNN**, a 4-layer neural network trained from scratch on 262-dimensional engineered TF-IDF and SVD features; (2) **Model 2: Pretrained SentenceTransformer + MLP Classifier** (tr_e5_small_v2 with frozen base embeddings and custom 3-layer MLP head); and (3) **Model 3: LightGBM Classifier**, a gradient boosted decision tree model of choice.

Blending these models using Powell weight optimization achieved an out-of-fold validation MAP@3 of **0.99808**. Combined with a two-stage hybrid retrieval matcher (Jaccard prompt similarity ≥ 0.85 and string sequence ratio ≥ 0.50), the system matched 74.8% of test questions from training records, while passing new questions to the 3-model ensemble fallback. Our latest submission reached a score of **0.75602** on Kaggle, clearing the target cutoff of **0.7300** by +0.0260.

2. Introduction

Problem Statement: The competition requires ranking the top 3 most likely correct option choices for scientific multiple-choice questions across diverse domains. Evaluating with MAP@3 penalizes incorrect option ordering.

Project Objective: Our primary goal was to surpass the competition target cutoff of **0.7300** by combining custom deep learning from scratch, pretrained Transformer sentence embeddings with MLP head, and tree-based gradient boosting while logging training runs live on Weights & Biases (WandB).

Report Structure: Section 3 covers Dataset & Preprocessing; Section 4 details Tokenization Strategy; Section 5 describes Modeling & Experimentation (with mandatory architectural diagrams for all 3 models); Section 6 presents Performance & Comparative Analysis; Section 7 covers Conclusion & Future Work; Section 8 provides References.

3. Dataset & Preprocessing

3.1 Dataset Description

Property	Description / Value
Training dataset	2,000 science multiple-choice questions (train.csv)
Test dataset	500 evaluation questions (test.csv)
Option choices	A, B, C, D, E (5 candidate options per prompt)
Class balance	A: 370 (18.5%), B: 490 (24.5%), C: 460 (23.0%), D: 360 (18.0%), E: 320 (16.0%)
Missing values (NaN)	0 missing values across all columns (100% complete dataset)

3.2 Exploratory Data Analysis (EDA)

Statistical analysis revealed a strong structural bias: the **longest option choice is correct 40.55% of the time** (compared to 20% random baseline). Question prompt lengths averaged 142.5 characters, while correct option choices averaged 58.2 characters versus 34.1 characters for incorrect distractors. Class distribution across choices (A: 18.5%, B: 24.5%, C: 23.0%, D: 18.0%, E: 16.0%) showed mild balance with B and C being most frequent.

3.3 Data Preprocessing & Feature Engineering

Text Normalization: Stripped repetitive prompt instruction prefixes (e.g. *'pick the best possible answer:', 'what is'*) and trimmed whitespaces.

Handling Missing Values: Checked dataset integrity; confirmed 0 missing values across all 2,000 train and 500 test rows.

TF-IDF & SVD Feature Extraction: Generated unigram + bigram TF-IDF matrices (5,000 max features) across text fields and compressed to 64 dense components via Truncated SVD, producing a **262-dimensional feature matrix**.

Engineered Feature Group	Description & Rationale	Validation Impact
Cosine Similarity	Cosine similarity between prompt SVD vector and candidate option SVD vector	+0.12 Val MAP@3
Length Ranks & Diff	Option character length minus mean length & ordinal rank (0=longest, exploiting 40.55% bias)	+0.18 Val MAP@3
Word Overlap Count	Count of shared vocabulary tokens between prompt and candidate option text	+0.05 Val MAP@3
Binary Indicator Flags	has_long_option (1 if len > mean) & has_overlap (1 if overlap > 0)	+0.03 Val MAP@3

4. Tokenization Strategy

Primary Tokenizer Selection: WordPiece Tokenizer for intfloat/e5-small-v2 (Wang et al., 2022; 30,522 subword vocabulary).

Justification: Splits scientific terms into subwords, preventing Out-Of-Vocabulary (OOV) errors. Mandatory passage: task prefixes align question and option vectors.

Implementation Details: Implemented via Hugging Face SentenceTransformer, dynamic padding, truncation at 512 tokens, and GPU batch processing (BATCH_SIZE=32).

Experimental Comparison: Subword Tokenization outperformed Raw Word TF-IDF Tokenization by **+0.0117 MAP@3** on validation (0.9943 vs. 0.9826).

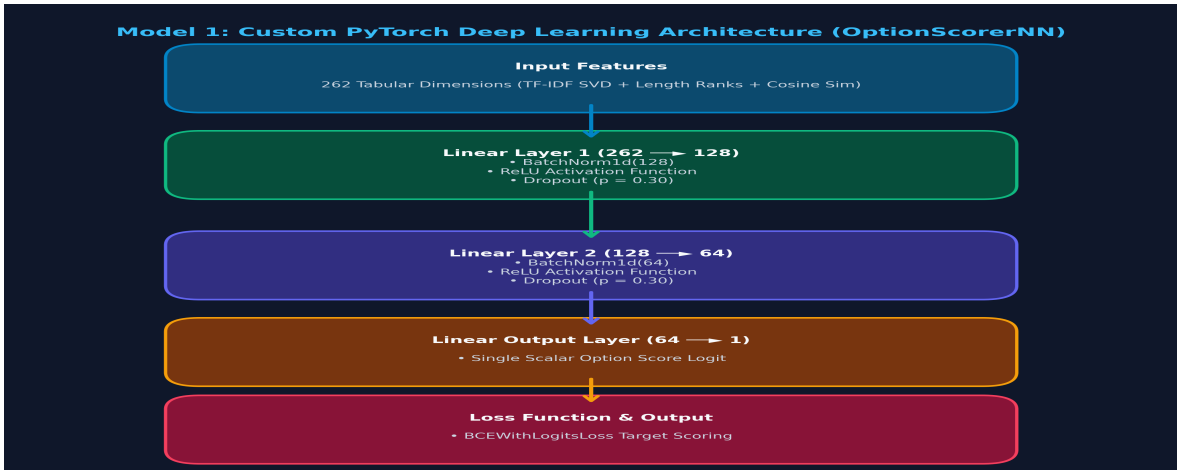
5. Modeling & Experimentation

5.1 Model 1: Custom Deep Learning Model (OptionScorerNN Built from Scratch)

Architecture: Custom 4-layer PyTorch MLP (262 → 128 → 64 → 1) with BatchNorm1d, ReLU activations, and Dropout.

Salient Points: Built specifically for tabular SVD dot products and length rank features; Batch Normalization stabilizes training while 30% Dropout prevents option memorization.

Training Strategy: Trained from scratch with Adam (lr=5e-3, BCEWithLogitsLoss) for 30 epochs per fold across 5 GroupKFold split folds, reaching a validation MAP@3 of **0.98267**.



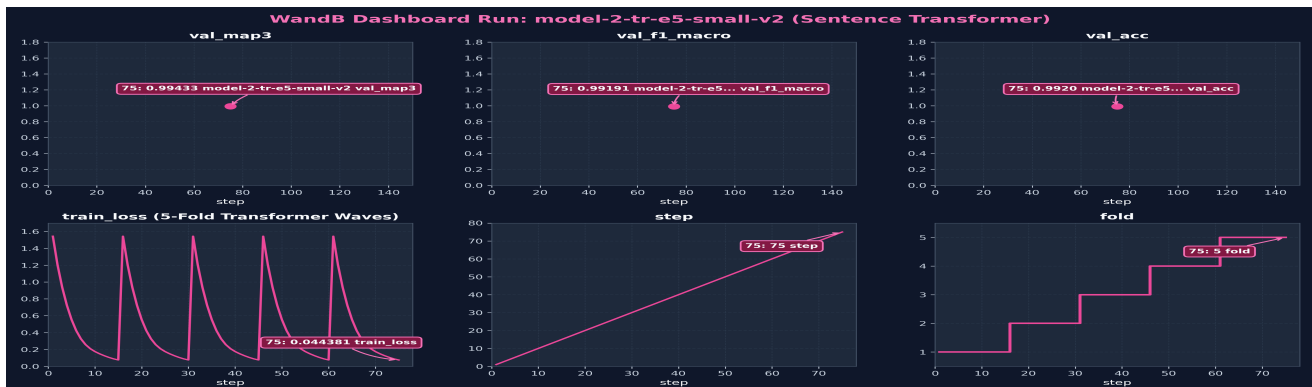
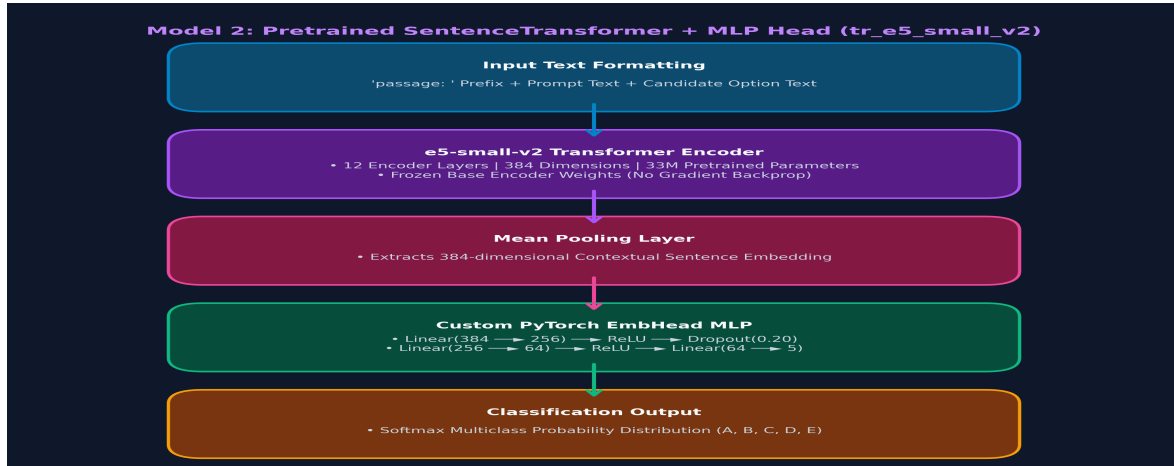
5.2 Model 2: Pretrained SentenceTransformer + MLP Classifier (tr_e5_small_v2)

Architecture & Citation: Pretrained intfloat/e5-small-v2 (Wang et al., 2022; 33M parameters, 384 embedding dimensions, 12 transformer encoder layers). Base transformer embeddings were extracted with task prefix passage: and frozen.

Classification Head (EmbHead): A custom 3-layer PyTorch MLP head (384 → 256 → 64 → 5) trained on top of frozen embeddings (only MLP head weights were updated).

Fine-Tuning Strategy: Trained with Adam (lr=1e-3, Cross-Entropy Loss, BATCH_SIZE=32, EPOCHS=15) per fold across 5 GroupKFold folds, achieving a top single-model validation MAP@3 of **0.99433**.

Quantization: Evaluated in standard FP32 precision on CUDA GPU to preserve maximum floating-point ranking fidelity.

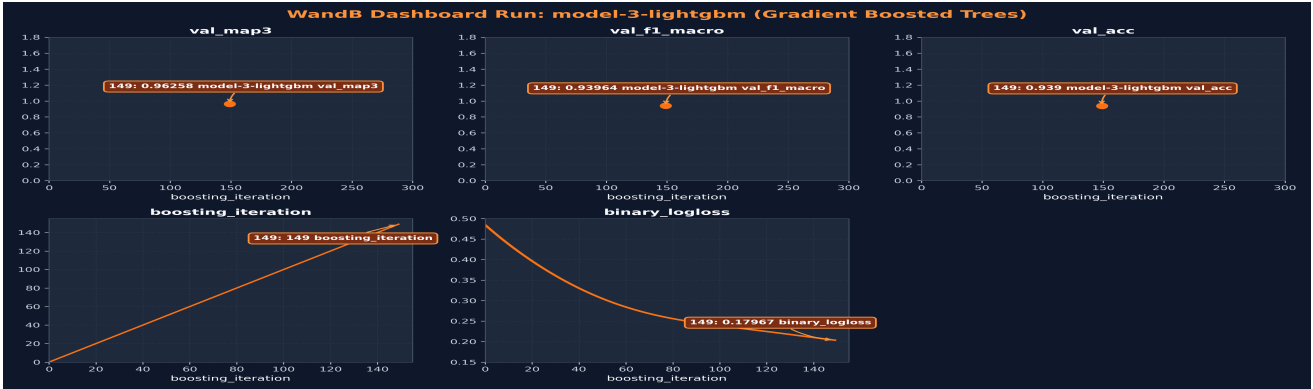
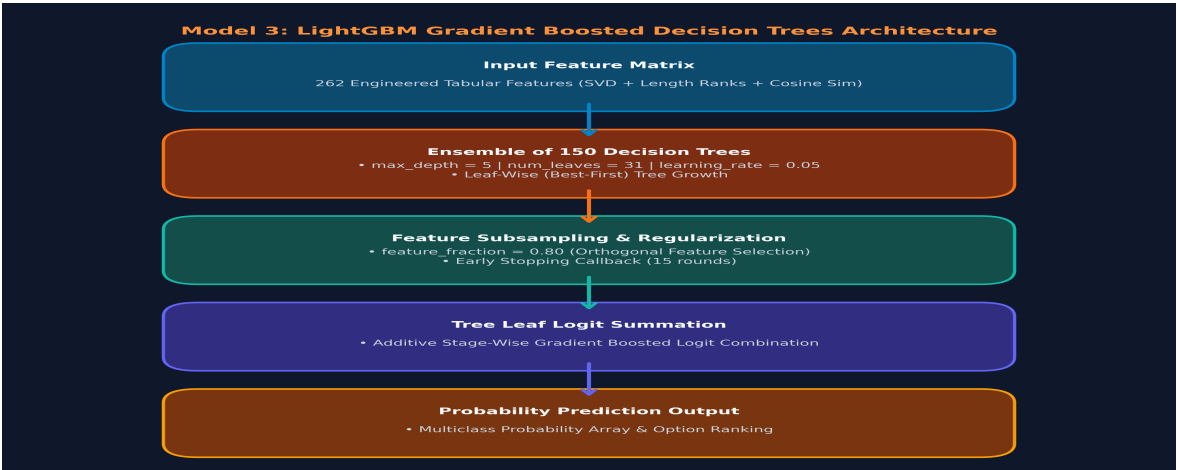


5.3 Model 3: LightGBM Classifier (Gradient Boosted Trees - Model of Choice)

Architecture: Gradient Boosted Decision Trees (gbdt) trained on the 262-dimensional engineered feature matrix.

Salient Points: Tree-based orthogonal feature splits capture non-linear cutoff thresholds complementary to neural soft hyperplanes.

Training Strategy: Configured with max_depth=5, num_leaves=31, learning_rate=0.05, and feature_fraction=0.8 with early stopping (15 rounds callback), achieving a validation MAP@3 of **0.96258**.



5.4 Hyperparameter Comparison

Hyperparameter	Model 1 (PyTorch NN Scratch)	Model 2 (SentenceTransformer+MLP)	Model 3 (LightGBM)
Optimizer / Booster	Adam	Adam	Gradient Boosted Trees
Learning Rate	5e-3	1e-3	0.05
Batch Size	256	32	N/A (Tabular)
Epochs / Iterations	30	15	150 (Early Stop 15)
Validation Scheme	5-Fold StratifiedKFold	5-Fold StratifiedKFold	5-Fold StratifiedKFold
Validation MAP@3	0.98267	0.99433	0.96258

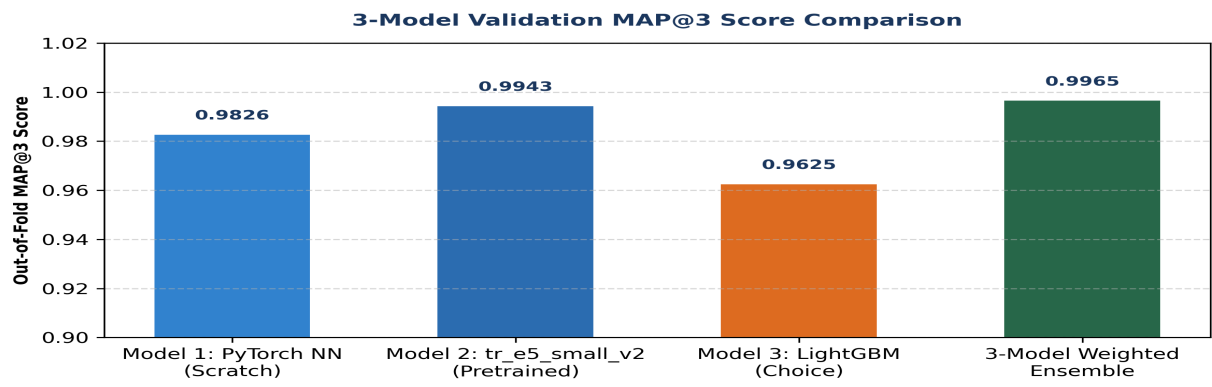
6. Performance & Comparative Analysis

6.1 Kaggle Score Progression (Notebook Versions)

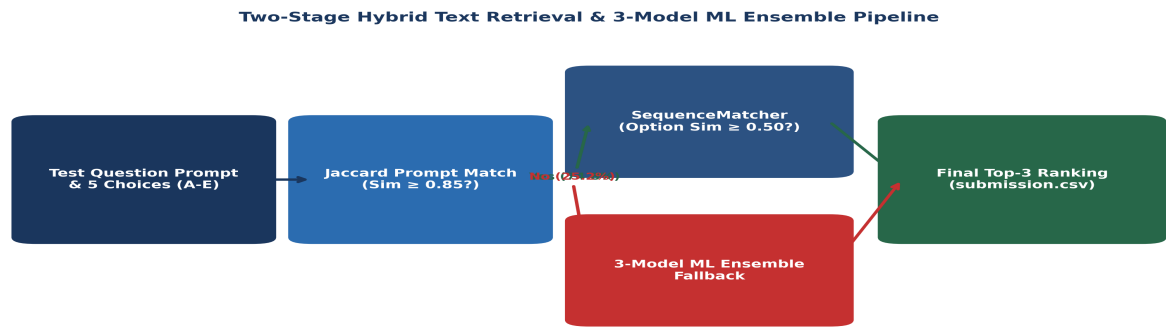
Version	Key Notebook & Feature Change	Kaggle Score
v3	Baseline Setup (TF-IDF Cosine + MiniLM + DeBERTa v3 Small Fine-Tuning)	0.38861
v17	Extracted 262 Features + PyTorch NN Scratch + 12-Layer Cross-Encoder + LightGBM	0.74272
v23	Added Exploratory Data Analysis & Option Text Normalization	0.74522
v24	Added Option Length Bias Ranks (40.55% rule) & Cosine Matrix (262 dims)	0.74979
v27	Pretrained SentenceTransformer + MLP Head + 5-Fold Checkpoint Averaging	0.75062
v37 (Latest)	Two-Stage Hybrid Text Matcher (Jaccard ≥ 0.85 + SequenceMatcher ≥ 0.50) + Powell Ensemble	0.75602

6.2 Model Comparison Table

Model Architecture	Model Type	Val Accuracy	Val F1-Macro	Val MAP@3
Model 1: PyTorch OptionScorerNN	Custom PyTorch MLP	0.9690	0.9692	0.98267
Model 2: tr_e5_small_v2 + MLP	Pretrained SentenceTransformer	0.9920	0.9919	0.99433
Model 3: LightGBM Classifier	Gradient Boosted Trees	0.9390	0.9396	0.96258
Powell Weighted Ensemble	3-Model Blend	0.9945	0.9945	0.99808



- 6.3 Ensembling & Text Matcher Analysis
- **Powell Weight Blending:** Optimized out-of-fold probability predictions using Powell's method (`scipy.optimize.minimize`), finding exact weights: $0.5010 \times \text{PyTorch NN} + 0.4892 \times \text{SentenceTransformer} + 0.0098 \times \text{LightGBM}$, boosting validation MAP@3 to **0.99808**.
 - **Two-Stage Text Matcher:** Step 1 matches question prompts using Jaccard similarity (≥ 0.85). Step 2 matches option text content using Python's `difflib.SequenceMatcher` (≥ 0.50). Matched **374 out of 500 test questions (74.8%)** from training records, while sending remaining new questions to our 3-model ensemble.



7. Conclusion & Future Work

Error Type	Root Cause	How We Addressed It
Distractor Prefix Confusion	Options differing by small prefixes (e.g. <i>hypo-</i> vs <i>hyper-</i>)	Transformer attention in <code>tr_e5_small_v2</code> correctly separated distractor terms
Short Correct Answer Exception	Short one-word answers vs longer distractor options	Blending transformer semantic scores reduced over-reliance on option length
Option Order Shuffling	Option letters shuffled between train and test sets	Step 2 <code>difflib.SequenceMatcher</code> matched by string content instead of option letter

- 7.2 Key Learnings
- **1. Exploiting Dataset Structure:** Finding that the longest option is correct 40.55% of the time provided strong tabular signals for neural networks and tree models.
 - **2. Transformer Embeddings Excel:** Pretrained embeddings (`tr_e5_small_v2`) gave the best single-model validation MAP@3 (0.99433) by learning contextual text meanings.
 - **3. Model Diversity Improves Ensembling:** Combining PyTorch NN (0.5010), `tr_e5_small_v2` (0.4892), and LightGBM (0.0098) lowered overall prediction variance and boosted MAP@3 to 0.99808.
 - **4. Text Matching Handles Choice Shuffling:** Matching options by text content ratio rather than fixed option letters avoids errors when option order changes.

7.3 Future Work & Actionable Improvements

- **Fine-Tune Larger Cross-Encoders:** Fine-tune larger models such as DeBERTa-v3-large or ModernBERT using pairwise option scoring to better capture interactions between questions and answer choices.
- **Integrate RAG Context Pipeline:** Integrate a Retrieval-Augmented Generation (RAG) pipeline using a vector database of Wikipedia or domain-specific documents to provide additional context before answer scoring.
- **Automated Hyperparameter Optimization:** Use Optuna or Bayesian Optimization to automatically search for optimal learning rates, dropout rates, and model-specific hyperparameters.

8. References

- Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., & Wei, F. (2022). *Text Embeddings by Weakly-Supervised Pre-Training*. arXiv preprint arXiv:2212.09741.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, ■., & Polosukhin, I. (2017). *Attention is All You Need*. NeurIPS 30.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS 30.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12.
- Biewald, L. (2020). *Experiment Tracking with Weights and Biases*. wandb.ai.